Day 63: The LSTM Cell — Gates Explained, Starting From Day 59's Problem

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 10, Day 1 | Complete study guide

Watch: Long Short-Term Memory (LSTM), Clearly Explained (StatQuest with Josh Starmer) | Illustrated Guide to LSTM's and GRU's: A step by step explanation (The AI Hacker)

Script: `day63_lstm_cell_gates.py` · every number in this guide is copied by program from `day63_run_output.txt`, the real output of that script.

Week 9 built the plain RNN and found its limit: it cannot carry information far. Week 10 replaces the cell. Today looks at **one LSTM cell doing one step**, and nothing more: what its four parts compute, what each gate is for, and whether it answers the problem Day 59 measured. Nothing is trained today. The main finding is not “the LSTM fixes vanishing gradients”. It is narrower and more useful: the LSTM adds **one new route** for memory whose strength is set by its **forget gate**, and with that gate at its plain default the cell forgets about as fast as a plain RNN (1.09e-04 of a nudge left after 20 steps, against 1.30e-04).

0. Where today fits

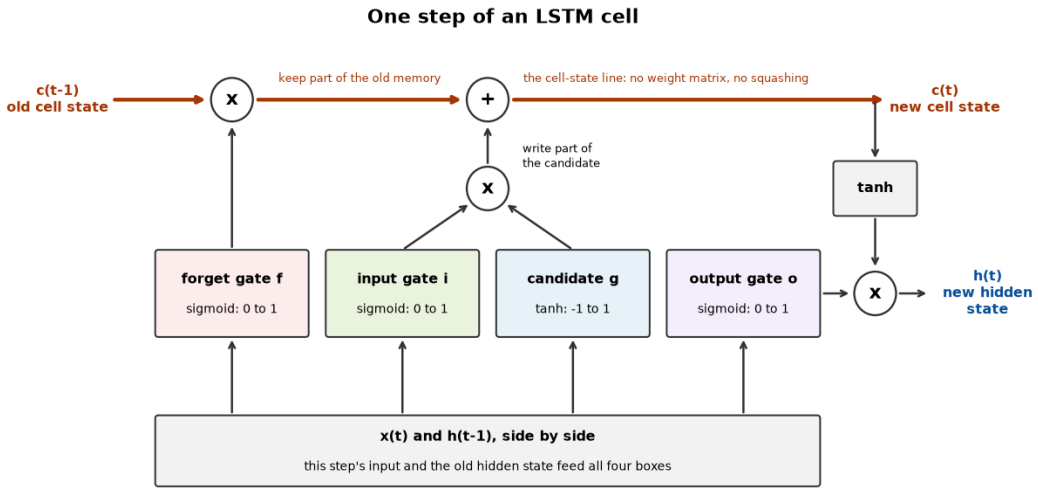
DEFINITION — start here, no prior knowledge assumed

An **LSTM** (Long Short-Term Memory) cell is a replacement for the RNN cell. It carries **two** states from one step to the next instead of one: the **cell state c**, a long-term memory, and the **hidden state h**, what the cell shows to the rest of the network at this step. Three **gates** — numbers between 0 and 1 — control the memory: how much to keep, how much to write, how much to show.

REAL-WORLD SCENARIO

A shared whiteboard in an office. Each morning three decisions are made: **what to wipe off** (forget gate), **what new note to write up** (input gate), and **what to read out at the meeting today** (output gate). A note that nobody wipes stays on the board for months without anyone having to repeat it. A plain RNN has no whiteboard: it must re-say everything it knows at every step, and what is re-said fifty times gets garbled.

SYSTEM DIAGRAM



One step of an LSTM cell. The thick line along the top is the cell state. The four boxes all read the same thing: this step's input and the previous hidden state.

How to read each section

Step	What it gives you
DEFINITION	What the idea is, in plain words. Start here.
REAL-WORLD SCENARIO	An everyday situation where the same idea shows up.
MATHEMATICS	The formula, when the section introduces one.
SPECIFICATION	The exact setup of the experiment: sizes, settings, what is measured.
Why this test	The question the experiment answers.
IMPLEMENTATION	The real code from the script that produces the result.
ACTUAL OUTPUT	What the script really printed, copied unchanged.
What it means	How to read the numbers, and what they do and do not show.
Why choose this? / Why NOT this?	When the idea is the right tool, and what it costs.

Key words used today

Word	Plain meaning
Cell state, c	The LSTM's long-term memory: a row of numbers carried from step to step.
Hidden state, h	What the cell shows at this step. Also fed back in at the next step.
Gate	A number between 0 and 1 that multiplies something: 0 blocks it, 1 lets all of it through.
Sigmoid	The function that produces a gate: it turns any number into one between 0 and 1.
Forget gate, f	How much of the old cell state to keep. (1 = keep everything; the name is misleading.)
Input gate, i	How much of the new candidate value to write into the cell state.
Candidate, g	The new value offered for writing, between -1 and 1.
Output gate, o	How much of the cell state to show as the hidden state.
Forget bias	A number added to the forget gate's raw score. Raising it pushes the gate toward 1.
Nudge sensitivity	Change the state a tiny bit at step 0; how big is the change at step T, per unit of nudge?

1. One step of an LSTM cell

DEFINITION — start here, no prior knowledge assumed

One step takes three things in — this step's input **x**, the old hidden state **h**, the old cell state **c** — and gives two things out: the new **h** and the new **c**. In between it computes four values from x and the old h: three gates (f, i, o) and one candidate (g). Then two short lines combine them.

A **sigmoid** squashes any number into the range 0 to 1, which is what makes it suitable for a gate. **tanh** squashes into -1 to 1, suitable for a value that can be positive or negative.

REAL-WORLD SCENARIO

You keep a running shopping list (the cell state). A message arrives: “We already have milk, but we're out of eggs.” You do three things: cross off milk (forget), write eggs (input), and when someone asks “what do I need from the dairy aisle?” you read out only the relevant part (output). The message and what you last said decide all three; the list itself is only crossed out and added to, never rewritten from scratch.

MATHEMATICS

$f = sigmoid([x, h_{old}] W_f + b_f)$ *forget gate, 0 to 1*

$i = sigmoid([x, h_{old}] W_i + b_i)$ *input gate, 0 to 1*

$g = tanh([x, h_{old}] W_g + b_g)$ *candidate, -1 to 1*

$o = sigmoid([x, h_{old}] W_o + b_o)$ *output gate, 0 to 1*

$c_{new} = f \cdot c_{old} + i \cdot g$ *(· means entry by entry)*

$h_{new} = o \cdot tanh(c_{new})$

$sigmoid(z) = 1 / (1 + e^{-z})$ *[x, h] means x and h placed side by side*

SPECIFICATION

Hand trace	One input, one hidden unit, so every quantity is a single number. x = 1.0, old h = 0.5, old c = -0.8. Eight weights and four biases chosen by hand.
Check 1	Every quantity is computed twice: once with plain arithmetic, once by `lstm_step_forward`.
Check 2	A random case with 5 examples, 3 inputs, 4 hidden units and random biases. The function (one stacked weight matrix) is compared with the textbook version (four separate gate formulas).
Pass criterion	Identical to rounding error (around 1e-16).

Why this test

The function stores all four sets of weights in **one** matrix and cuts the result into four blocks. That is fast and is how real libraries do it, but it is easy to slice the wrong block or put the gates in the wrong order. Days 64 and 65 will loop this function over time and differentiate through it, so it is checked here against the formulas written the long way. There is no gradient to check yet — that is Day 65.

IMPLEMENTATION

```
def sigmoid(z):
    return 1.0 / (1.0 + np.exp(-z))
def lstm_step_forward(x_t, h_prev, c_prev, W, b):
    H = h_prev.shape[1]
    a = np.concatenate([x_t, h_prev], axis=1) @ W + b
    f = sigmoid(a[:, 0 * H:1 * H])
    i = sigmoid(a[:, 1 * H:2 * H])
    g = np.tanh(a[:, 2 * H:3 * H])
    o = sigmoid(a[:, 3 * H:4 * H])
    c = f * c_prev + i * g
    h = o * np.tanh(c)
    return h, c, (f, i, g, o)
```

The subtle part: a has 4 × H columns. Columns 0 to H are the forget gate's raw scores, the next H the input gate's, then the candidate's, then the output gate's. The order f, i, g, o is a convention of this series and must stay the same all week.

ACTUAL OUTPUT

```
inputs: x = 1.0, old hidden state h = 0.5, old cell state c = -0.8
quantity                    raw score    by hand   function
forget gate f (sigmoid)        1.1000     0.7503     0.7503
input gate i (sigmoid)        -0.8000     0.3100     0.3100
candidate g (tanh)             1.7000     0.9354     0.9354
output gate o (sigmoid)        1.3000     0.7858     0.7858
new cell state c                                   -0.3102    -0.3102
new hidden state h                                 -0.2362    -0.2362
  new c = f * old c + i * g = 0.7503 * (-0.8) + 0.3100 * 0.9354 = -0.3102
  new h = o * tanh(new c)   = 0.7858 * (-0.3006) = -0.2362
```

```
Actual output from this lesson's run.
```

ACTUAL OUTPUT

```
random check (5 examples, 3 inputs, 4 hidden units): four separate gate formulas vs. the function
    -- max difference in c 5.55e-17, in h 6.94e-17
Actual output from this lesson's run.
```

What it means

Follow the numbers. The forget gate came out at 0.7503, so about three quarters of the old memory (-0.8) is kept. The input gate is 0.3100, so under a third of the candidate (0.9354) is written. New memory: -0.3102. The output gate (0.7858) then shows most of the squashed memory: h = -0.2362. The hand arithmetic and the function agree on every line.

On the random case the stacked-matrix function and the four separate formulas differ by at most 5.55e-17, which is rounding. The cell is implemented correctly.

WHY CHOOSE THIS?

One matrix product computes all four parts at once, and the two update lines are simple multiply-and-add. The whole cell is nine lines of code.

WHY NOT THIS? (TRADEOFFS)

It is more to hold in your head than the RNN's single line, and it has four times the weights (Section 5). Whether the extra parts earn their keep is what the rest of today tests.

2. What each gate does

DEFINITION — start here, no prior knowledge assumed

A **gate** is a multiplier between 0 and 1. Multiplying by 0 blocks something completely; multiplying by 1 lets all of it through; 0.5 lets half through. The LSTM has three: **f** multiplies the old memory, **i** multiplies the new candidate, **o** multiplies what is shown. Here the gates are set directly to chosen values, with no weights involved, to see each one's job on its own.

REAL-WORLD SCENARIO

Three taps on three pipes. The first tap is on the pipe carrying yesterday's water into today's tank (forget gate: closed means yesterday's water is thrown away). The second is on the pipe bringing fresh water in (input gate). The third is on the outlet (output gate: closed means the tank is full but nothing comes out). Each tap can be anywhere between fully closed and fully open.

MATHEMATICS

$$c_{new} = f \cdot c_{old} + i \cdot g \qquad h_{new} = o \cdot \tanh(c_{new})$$

$$f = 1,\ i = 0 \ \rightarrow\ c_{new} = c_{old} \text{ exactly: the memory is copied forward untouched}$$

SPECIFICATION

Fixed	Old cell state c = 0.80; candidate g = -0.50.
Varied	The three gates f, i, o, set by hand to 0, 1 or 0.5 in six combinations.
Measured	The new cell state and the new hidden state.
Note	In a real LSTM the gates are computed from x and h and are rarely exactly 0 or 1. This is the mechanism on its own.

Why this test

The six formulas of Section 1 say what is computed but not what it is *for*. Setting the gates to their extreme values turns the cell into something familiar: a memory that can be kept, erased, overwritten, added to, or hidden. Every behaviour of a trained LSTM is a blend of these.

IMPLEMENTATION

```
def cell_update(c_prev, f, i, g, o):
    c = f * c_prev + i * g
    return c, o * np.tanh(c)
```

ACTUAL OUTPUT

```
what the gates say                    f    i    o    c_new    h_new
keep the memory, show it              1    0    1    0.800    0.664
keep the memory, hide it              1    0    0    0.800    0.000
erase the memory                      0    0    1    0.000    0.000
replace it with the candidate         0    1    1   -0.500   -0.462
add the candidate to it               1    1    1    0.300    0.291
half of each                        0.5  0.5    1    0.150    0.149
Actual output from this lesson's run.
```

What it means

- **Keep** (f = 1, i = 0): the memory stays at 0.800. This is the case that matters most for long sequences, and the RNN has no equivalent.
- **Hide** (o = 0): the memory is still 0.800 but the hidden state is 0.000. The cell can remember something without showing it. In an RNN, state and output are the same thing.
- **Erase** (f = 0, i = 0) and **replace** (f = 0, i = 1): the old value is gone in one step.
- **Add** (f = 1, i = 1): 0.300 = 0.80 + (-0.50). The cell state can count or accumulate; it is not limited to the range -1 to 1.
- **The forget gate is badly named.** f = 1 means *remember*. Read it as a “keep” gate.

WHY CHOOSE THIS?

Separate dials for keeping, writing and showing let one cell behave as a store, an eraser, a counter or a blend, and switch between them from step to step.

WHY NOT THIS? (TRADEOFFS)

The dials are not set by you. They are computed from the input and the previous hidden state through learned weights, so the network has to learn when to open and close them.

3. Store, hold, read: a memory cell with hand-set weights

DEFINITION — start here, no prior knowledge assumed

Hand-set weights means choosing the weights yourself instead of learning them, to prove that a behaviour is *possible* for the cell. Here one LSTM unit is wired to **store** a number when told to, **hold** it while unrelated numbers stream past, and **read** it out when asked.

The **sharpness k** says how firmly the gates are pushed: a gate's raw score is +k/2 when it should be open and -k/2 when it should be shut. A sigmoid never gives exactly 0 or 1, so a gate always **leaks** a little; bigger k means a smaller leak.

REAL-WORLD SCENARIO

A left-luggage locker. You put a bag in and lock it (write). Hundreds of people walk past with their own bags and none of them changes what is in your locker (hold). You come back with the key and take it out (read).

The leak is like a phone that loses 0.7% of its charge every hour when switched off. You would never notice in an afternoon. After six weeks it is flat. “99.3% kept per step” sounds safe and is not, if there are a thousand steps.

MATHEMATICS

$$\textit{open gate} = \sigma(+k/2), \quad \textit{shut gate} = \sigma(-k/2) \qquad k = 10 \rightarrow 0.9933 \text{ and } 0.0067$$

$$\textit{memory left after } T \textit{ steps of holding} \approx f^{T} \qquad 0.9933^{999} = 0.0012$$

SPECIFICATION

Cell	One LSTM unit. Three inputs per step: a value, a write flag (0 or 1), a read flag (0 or 1).
Weights	Set by hand: forget gate shut only when write = 1; input gate open only when write = 1; output gate open only when read = 1; candidate = tanh(value).
Sequence	Step 0 writes a value v drawn from -1 to 1. Each later step carries a distracting random value (standard deviation 0.5) with both flags at 0. The last step raises the read flag.
Varied	Sharpness k = 10, 20, 40; held for 9, 99 or 999 steps.
Measured	Over 1,000 independent trials: mean distance between the final cell state and what was stored; their correlation; the largest hidden-state value before the read step.
Contrast	The simplest hand-set RNN memory: one unit, h = tanh(h + value), recurrent weight 1, with and without the distractors. This is not a trained RNN.

Why this test

Section 2 set the gates directly. A real cell cannot do that; it only has weights. This test asks whether weights exist that make the gates do the right thing at the right time, and how exact they have to be. It also separates two things a memory must do that are easy to confuse: **not fading** and **not being overwritten** by whatever arrives next.

IMPLEMENTATION

```
def handset_memory_cell(k):
    W = np.zeros((4, 4))
    b = np.zeros(4)
    W[1, 0], b[0] = -k, k / 2
    W[1, 1], b[1] = k, -k / 2
    W[0, 2] = 1.0
    W[2, 3], b[3] = k, -k / 2
    return W, b
# inside demo_store_hold_read(): build the sequences and run the cell
X = np.zeros((n, T, 3))
X[:, 0, 0], X[:, 0, 1] = v, 1.0
X[:, 1:, 0] = noise[:, :T - 1]
X[:, -1, 2] = 1.0
h, c = np.zeros((n, 1)), np.zeros((n, 1))
hidden_peak, trace = 0.0, []
for t in range(T):
    h, c, _ = lstm_step_forward(X[:, t, :], h, c, W, b)
```

The subtle part: the write flag drives two gates in opposite directions. Its weight to the input gate is +k and to the forget gate is -k, so writing opens one and shuts the other.

ACTUAL OUTPUT

```
   k    open gate   shut gate  steps held  mean |c - stored|  correlation  max |h| before read
  10  0.993307149    6.69e-03           9           2.92e-02     0.999821             4.28e-03
  10  0.993307149    6.69e-03          99           2.15e-01     0.996893             4.28e-03
```

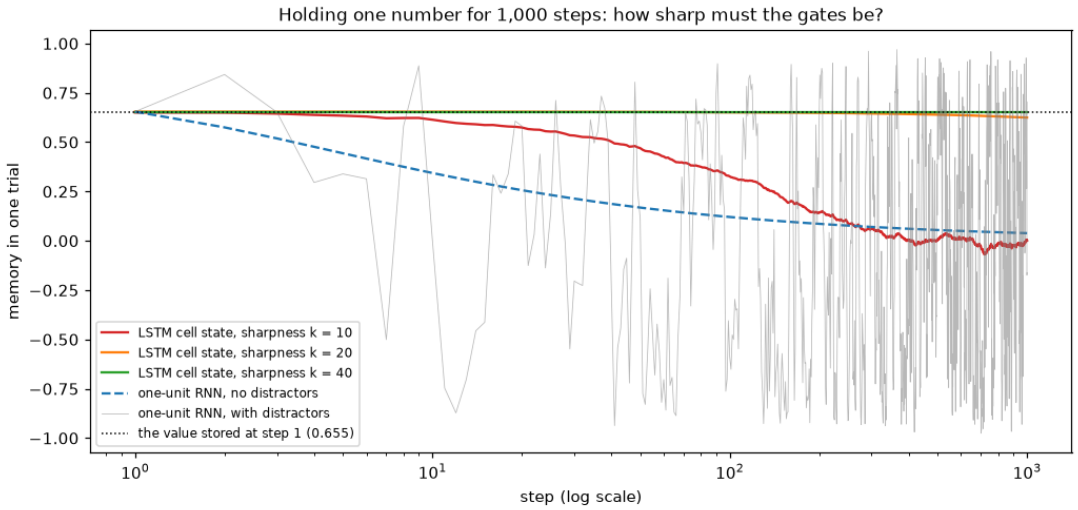
```
  10  0.993307149    6.69e-03         999           4.36e-01     0.037012             4.28e-03
  20  0.999954602    4.54e-05           9           2.04e-04     1.000000             2.91e-05
  20  0.999954602    4.54e-05          99           1.99e-03     1.000000             2.91e-05
  20  0.999954602    4.54e-05         999           1.94e-02     0.999999             2.91e-05
  40  0.999999998    2.06e-09           9           9.26e-09     1.000000             1.32e-09
  40  0.999999998    2.06e-09          99           9.07e-08     1.000000             1.32e-09
  40  0.999999998    2.06e-09         999           9.02e-07     1.000000             1.32e-09
Actual output from this lesson's run.
```

ACTUAL OUTPUT

```
prediction for the leak: a forget gate of value f keeps f^T of the memory after T steps.
   k            f       f^9      f^99     f^999
  10  0.993307149    0.9414    0.5144    0.0012
  20  0.999954602    0.9996    0.9955    0.9557
  40  0.999999998    1.0000    1.0000    1.0000
Actual output from this lesson's run.
```

ACTUAL OUTPUT

```
for contrast, the simplest hand-set RNN memory: one unit, h = tanh(h + value), recurrent weight 1.
    It has no gates, so every value is added in.
 steps held   distractors: corr   no distractors: corr   no distractors: 0.9 becomes
          9              0.0177                0.9846                        0.3515
         99             -0.0628                0.9420                        0.1210
        999             -0.0030                0.9108                        0.0387
Actual output from this lesson's run.
```



One trial, 1,000 steps (log scale). The dotted line is the stored value. Green and orange (k = 40, 20) stay on it; red (k = 10) leaks away; the one-unit RNN either shrinks (dashed) or is thrown around by the distractors (grey).

What it means

It can be done. At k = 40 the cell holds 1,000 different numbers through 999 distracting steps to within 9.02e-07 on average, and gives nothing away before it is asked (largest hidden value 1.32e-09).

“Nearly open” is not enough. At k = 10 the forget gate is 0.993307149: 99.3% open. After 99 steps the stored value is already off by 2.15e-01 on average, and after 999 steps its correlation with what was stored is 0.037012: the memory is gone. The prediction f^{999} = 0.0012 says the same. At k = 20 the gate is 0.99995 and 95.6% survives 999 steps. How long an LSTM remembers is set by how close its forget gate gets to 1.

The one-unit RNN fails in two different ways. With distractors, its state has a correlation of 0.0177 with the stored value after only 9 steps: it has no way to refuse an input, so every new value is added in. With no distractors at all, the order of values is kept (correlation 0.9108 after 999 steps) but their size is not: a stored 0.9 has become 0.0387, because tanh shrinks the state a little at every step.

What this is not. The RNN here is one unit with one hand-chosen weight, set against a hand-built LSTM. It shows what each *mechanism* can and cannot do. Day 58 showed that a trained multi-unit RNN can latch onto large signals. A fair contest between trained networks is Day 67.

WHY CHOOSE THIS?

With its gates firmly set, one LSTM unit is an exact, addressable memory: it ignores what it is told to ignore and keeps a real number, not just a sign, for a thousand steps.

WHY NOT THIS? (TRADEOFFS)

The gates have to be driven very close to 0 and 1, which means large weights or biases, and a sigmoid that saturated has a slope near zero, which makes it slow to adjust by training.

4. Day 59's question, asked of the LSTM

DEFINITION — start here, no prior knowledge assumed

Nudge sensitivity: change the state by a tiny amount at step 0, run the network forward T steps, and measure how big the change has become, per unit of nudge. 1 means the nudge arrived intact; far below 1 means it faded away; far above 1 means it grew. This is the forward-looking twin of Day 59's gradient measurement: if a change at step 0 cannot reach step T, then an error at step T cannot send a correction back to step 0.

In an LSTM a nudge to the cell state travels by two routes. The **direct route** is the cell-state line itself, where at each step the nudge is simply multiplied by that step's forget gate. The **indirect route** goes through the hidden state and the gates, like the RNN's only route.

REAL-WORLD SCENARIO

The whispering game with 50 players. In the RNN version each player hears the message and must say it again in their own words; after 50 retellings it is either a mumble or a wild exaggeration.

The LSTM adds a **written note** passed down the line alongside the whispering. Each player may tear a strip off the note before passing it on (the forget gate). If everybody tears off half, nothing is left after a few players. If everybody passes it on almost whole, it arrives readable. The whispering still goes on as well, and can still distort things.

MATHEMATICS

RNN, one step back: $(1 - h^2)$ *and by* W_{hh} — *a matrix and a squashing slope, every step*

LSTM, direct route, one step back: f_t — *just the forget gate, entry by entry*

direct route over T steps $= f_1 \cdot f_2 \cdot \ldots \cdot f_T$ — *never above 1; near 1 only if every f is near 1*

forget gate at the start of training $\approx$ *sigmoid(forget bias)* — *bias* $0 \to 0.5$, $1 \to 0.73$, $2 \to 0.88$, $4 \to 0.98$

SPECIFICATION

Setup	16 units; 200 sequences whose inputs are all N(0,1), as on Day 59. All states start at zero. Nothing is trained.
RNN	Day 59's seed-3 network, its recurrent matrix rescaled to spectral radius 0.5, 1.0, 1.5, 2.0, 3.0. The hidden state at step 0 is nudged; the hidden state at step T is measured.
LSTM	One set of random weights (seed 3); only the forget gate's bias differs: 0, 1, 2, 4. The cell state at step 0 is nudged; the cell state at step T is measured.
Lengths	T = 5, 10, 20, 50, 100 in the tables; every T from 1 to 100 in the plot.

Measurement	Central difference with a nudge of 1e-4 in a fixed random direction. Values below 1e-9 are printed as “<1e-09”: too small for this method to measure reliably.
Tool check	An LSTM whose gates are cut off from h has no indirect route, so its sensitivity must equal the product of its forget gates exactly.
Also	The direct route alone (the product of forget gates, which is exact), and the LSTM with every weight multiplied by 3 and by 10.

Why this test

The usual one-line claim is “LSTMs solve the vanishing gradient problem”. Today's cell is simple enough to check what is actually true. Three questions: does a nudge survive in an LSTM where it died in the RNN? What does that depend on? And can an LSTM still blow up?

IMPLEMENTATION

```
def rnn_final_state(h0, X, Wxh, Whh):
    h = h0
    for t in range(X.shape[1]):
        h = rnn_step_forward(X[:, t:t + 1], h, Wxh, Whh, 0.0)
    return h
def lstm_final_cell(c0, X, W, b):
    h, c = np.zeros_like(c0), c0
    kept = np.ones_like(c0)
    for t in range(X.shape[1]):
        h, c, (f, _, _, _) = lstm_step_forward(X[:, t:t + 1], h, c, W, b)
        kept = kept * f
    return c, kept
def nudge_sensitivity(run, state0, direction, eps=1e-4):
    change = run(state0 + eps * direction) - run(state0 - eps * direction)
    return np.linalg.norm(change) / (2 * eps)
# inside demo_memory_survival(): the check on a case with a known answer
W[1:, :] = 0.0
X = np.random.RandomState(0).randn(m, 20)
kept = lstm_final_cell(zero, X, W, b)[1]
measured = nudge_sensitivity(lambda c0: lstm_final_cell(c0, X, W, b)[0], zero, direction)
exact = np.linalg.norm(kept * direction)
```

The subtle part: `kept = kept * f` multiplies the forget gates together as the cell runs. That product *is* the direct route, with no measurement needed.

ACTUAL OUTPUT

```
check of the measuring tool (gates cut off from h, so the exact answer is known): measured
    0.0764194987, exact 0.0764194987, difference 2.1e-13
Actual output from this lesson's run.
```

ACTUAL OUTPUT

```
vanilla RNN -- nudge the hidden state at step 0, measure the hidden state at step T:
    T     rho=0.5     rho=1.0     rho=1.5     rho=2.0     rho=3.0
    5    6.43e-03    1.40e-01    6.79e-01    2.01e+00    8.39e+00
   10    3.71e-05    1.77e-02    3.79e-01    2.70e+00    3.15e+01
   20    1.18e-09    1.30e-04    5.53e-02    4.16e+00    2.83e+02
   50      <1e-09      <1e-09    2.22e-03    2.78e+00    4.40e+04
  100      <1e-09      <1e-09    4.31e-09    8.70e-02    1.45e+05
Actual output from this lesson's run.
```

ACTUAL OUTPUT

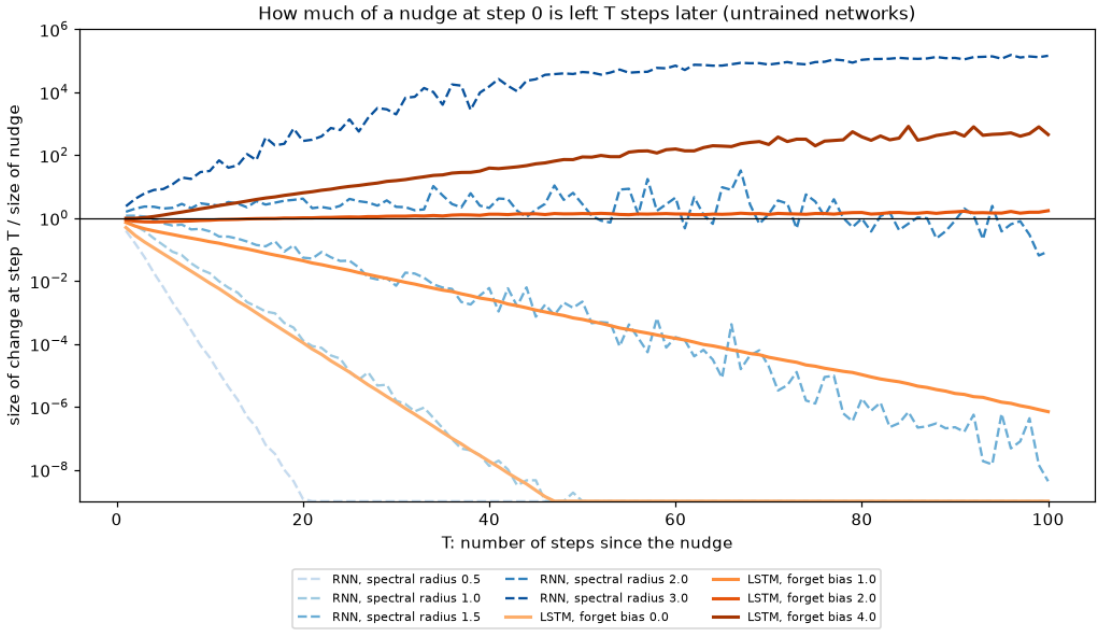
```
LSTM -- nudge the cell state at step 0, measure the cell state at step T:
    T    f-bias=0.0    f-bias=1.0    f-bias=2.0    f-bias=4.0
    5      7.35e-02      3.42e-01      7.65e-01      1.25e+00
   10      8.58e-03      1.78e-01      8.57e-01      2.20e+00
   20      1.09e-04      4.42e-02      1.02e+00      6.36e+00
   50        <1e-09      6.03e-04      1.39e+00      8.78e+01
  100        <1e-09      7.03e-07      1.71e+00      4.53e+02
Actual output from this lesson's run.
```

ACTUAL OUTPUT

```
LSTM, the direct path only: the product of the forget gates (no nudge needed; this part is exact):
    T    f-bias=0.0    f-bias=1.0    f-bias=2.0    f-bias=4.0
    5      3.26e-02      2.09e-01      5.27e-01      9.12e-01
   10      1.05e-03      4.39e-02      2.79e-01      8.32e-01
   20      1.19e-06      2.02e-03      8.05e-02      6.93e-01
   50      1.63e-15      2.10e-07      2.08e-03      4.00e-01
  100      4.06e-30      6.59e-14      6.74e-06      1.68e-01
    f         0.500         0.731         0.881         0.982   <- sigmoid(f-bias)
Actual output from this lesson's run.
```

ACTUAL OUTPUT

```
can the LSTM blow up? forget bias 4, T = 50, every weight multiplied by a factor:
 weights x  nudge sensitivity  direct path
       1.0           8.78e+01     4.00e-01
       3.0           1.71e+02     3.86e-01
      10.0           2.80e+05     1.16e-01
Actual output from this lesson's run.
```



Dashed blue: the RNN at five spectral radii. Solid orange: the LSTM at four forget biases. The black line at 1 means "the nudge arrived unchanged". Curves are cut off at 1e-9.

What it means

The measuring tool is right. On the case with a known answer it gives 0.0764194987, the exact value, to within 2.1e-13.

The RNN: fade or blow up. After 50 steps the nudge is below 1e-9 at spectral radius 0.5 and 1.0, 2.22e-03 at 1.5, 2.78e+00 at 2.0 and 4.40e+04 at 3.0. This is the same picture as Day 59's gradient measurement on the same networks (7.43e-23, 2.02e-10, 5.06e-04, 3.39e+00, 5.20e+04): one number, the spectral radius, decides, and the safe band between fading and exploding is narrow.

An LSTM with forget bias 0 is no better. Its forget gates sit near 0.5, so the direct route halves at every step: 1.19e-06 after 20 steps, 1.63e-15 after 50. The total sensitivity after 20 steps is 1.09e-04, about the same as the RNN at spectral radius 1.0 (1.30e-04). The LSTM does not prevent vanishing by being an LSTM.

Raising the forget bias changes everything. At bias 4 the gates are near 0.98 and 4.00e-01 of the nudge is still travelling the direct route after 50 steps, 1.68e-01 after 100. The RNN has no such route: everything it carries passes through the weight matrix and tanh at every step.

The direct route cannot explode; the indirect one can. The product of forget gates is never above 1. Yet the total sensitivity at bias 4 reaches 8.78e+01 at 50 steps and 4.53e+02 at 100, and with weights ten times larger, 2.80e+05 while the direct route is only 1.16e-01. All of that growth comes through the hidden state and the gates. LSTMs still need gradient clipping.

A number that looks better than it is. At bias 2 the total stays between 7.65e-01 and 1.71e+00 over every length — apparently perfect. But the direct route at 100 steps is only 6.74e-06. Almost none of that "1.71" is the original memory arriving down the cell-state line; it is the indirect route happening to balance near 1 with these particular random weights. A sensitivity near 1 is not, by itself, proof that memory is being kept.

READ THIS CAREFULLY

What is true, then. The LSTM does not remove the vanishing problem. It adds a route whose strength is a **dial** — the forget gate — that is bounded by 1, separate for every unit, and learnable. The practical consequence, well known in the literature, is to start training with the forget bias at 1 or 2 instead of 0, so the dial starts open (Gers, Schmidhuber & Cummins, 2000; Jozefowicz, Zaremba & Sutskever, 2015). The forget gate itself was added in 2000; the original 1997 LSTM kept everything.

WHY CHOOSE THIS?

The cell-state line gives memory a route with no weight matrix and no squashing on it, controlled by one gate per unit. With the gate near 1, information and gradient can cross 100 steps.

WHY NOT THIS? (TRADEOFFS)

Only if the gate is near 1: at the default bias the cell forgets as fast as an RNN. The indirect route still fades or grows as before, so an LSTM can still explode. And today's networks are untrained: whether training keeps the gates where they are needed is not shown here.

5. What the cell costs

DEFINITION — start here, no prior knowledge assumed

The **number of weights** (parameters) is how many separate numbers training has to set. More weights means more memory, more arithmetic at every step, and more data needed to set them well. An LSTM computes four things where an RNN computes one, each with its own full set of weights.

REAL-WORLD SCENARIO

A one-person shop against a shop with a manager, a buyer, a stock-keeper and a salesperson. The second shop can do more, but it has four salaries to pay and every decision passes through more hands. For a market stall that is too much; for a department store it is the least you need.

MATHEMATICS

$$\text{RNN weights} = (D + H + 1) \cdot H \qquad D = \text{input size}, H = \text{hidden size}$$

$$\text{LSTM weights} = 4 \cdot (D + H + 1) \cdot H \qquad \text{the same block, once each for } f, i, g, o$$

SPECIFICATION

Weights	Counted from the actual arrays for three sizes: Day 59's (1 input, 16 hidden), Day 62's tagger (32, 32), and a larger one (64, 128).
Time	20,000 single steps of each cell with input 32, hidden 32, batch 32, timed with `time.perf_counter`.
Caveat	Timings depend on the machine and change from run to run. The weight counts do not.

Why this test

Sections 3 and 4 showed what the extra machinery can do. A fair account also says what it costs, measured, before the next days start training it.

IMPLEMENTATION

```
n_rnn = D * H + H * H + H
start = time.perf_counter()
for _ in range(20000):
    lstm_step_forward(x, h, c, W, b)
t_lstm = time.perf_counter() - start
```

ACTUAL OUTPUT

```
 input  hidden  RNN weights  LSTM weights  ratio
     1      16          288          1152    4.0
    32      32         2080          8320    4.0
    64     128        24704         98816    4.0
time for 20,000 steps (input 32, hidden 32, batch 32): RNN 0.46 s, LSTM 1.57 s, ratio 3.4
    (timings vary from run to run and machine to machine)
Actual output from this lesson's run.
```

What it means

The weight count is exactly four times the RNN's at every size: 2080 against 8320 at Day 62's tagger size. In this run one LSTM step took about 3.4 times as long as one RNN step. That ratio moved between runs while this lesson was being built, so treat it as “a few times slower”, not as a constant.

WHY CHOOSE THIS?

The cost is a fixed factor of four in weights, not something that grows with sequence length, and it buys a memory route the RNN does not have.

WHY NOT THIS? (TRADEOFFS)

On a small dataset four times the weights means four times as much to overfit with. Day 61's sentiment RNN already memorised its 2,000 training sentences; a larger cell would not obviously help there.

6. What today did not show

Every gate in this lesson was either set by hand or left at random values. That is enough to establish what an LSTM cell **can** do and what its memory depends on. It does not establish that training **finds** good gate settings, or that an LSTM beats an RNN on any real task. Those are separate claims and each has its own day:

- **Day 64**: the forward pass over whole sequences — today's function inside a loop.
- **Day 65**: the backward pass and a gradient check, after which an LSTM can be trained.
- **Day 67**: trained RNN, LSTM and GRU compared on Week 9's data.

Interview preparation — technical questions

Q: Write the equations of an LSTM step.

A: f = sigmoid([x, h] Wf + bf); i = sigmoid([x, h] Wi + bi); g = tanh([x, h] Wg + bg); o = sigmoid([x, h] Wo + bo); c_new = f * c_old + i * g; h_new = o * tanh(c_new). The products in the last two lines are entry by entry.

Q: How many parameters does an LSTM layer have?

A: 4 * (D + H + 1) * H for input size D and hidden size H: four blocks (three gates and the candidate), each with weights from the input, weights from the hidden state, and a bias. Exactly four times a vanilla RNN.

Q: Why are the gates sigmoids and the candidate a tanh?

A: A gate is a fraction to let through, so it must lie between 0 and 1: sigmoid. The candidate is a value to store and should be able to be positive or negative: tanh, between -1 and 1.

Q: What should the forget gate's bias be initialised to, and why?

A: A positive value such as 1 or 2. At bias 0 the gate starts near 0.5 and the cell state halves every step; here the direct route was 1.19e-06 after 20 steps. At bias 4 it was still 0.168 after 100. Starting open lets gradient reach early steps from the beginning of training.

Q: Do LSTMs need gradient clipping?

A: Yes. The direct cell-state route is bounded by 1, but the route through the hidden state and the gates is not. With large weights the nudge sensitivity here reached 2.8e+05 while the direct route was 0.12.

Q: How would you verify an LSTM step implementation before training?

A: Trace one step by hand on a one-unit cell; compare the stacked-matrix version against the gate-by-gate formulas on random inputs (difference near 1e-16); then, once the backward pass exists, a numeric gradient check.

Interview preparation — theoretical questions

Q: Does the LSTM solve the vanishing gradient problem?

A: Not unconditionally. It adds a path, the cell state, along which the factor per step is the forget gate instead of a weight matrix times a tanh slope. If the forget gates are near 1 that path preserves signal over long distances; if they are near 0.5 it vanishes as fast as in an RNN. The other paths behave as before.

Q: What is the difference between the cell state and the hidden state?

A: The cell state is the internal memory, updated only by a multiply and an add, and not limited to -1..1. The hidden state is a gated, squashed view of it: what the cell shows to the next layer and feeds back to its own gates. A cell can hold something in c while showing nothing in h.

Q: Why can an RNN not simply use a recurrent weight of 1 to remember?

A: Two reasons, both measured today on a one-unit example. tanh shrinks the state a little every step (0.9 became 0.039 after 999 steps), and with no input gate every new input is added to the state, so the stored value is overwritten (correlation 0.02 after 9 steps).

Q: Why is a saturated gate both good and bad?

A: Good for memory: only a gate very close to 1 holds a value for hundreds of steps. Bad for learning: a saturated sigmoid has a slope near zero, so the gradient that would adjust that gate is tiny.

Q: What did the original 1997 LSTM lack?

A: A forget gate. The cell state was always carried forward in full; the forget gate was added by Gers, Schmidhuber and Cummins in 2000 so that a cell could reset itself.

Key takeaway and what's next

> **KEY TAKEAWAY**
> An LSTM cell keeps a memory, the cell state, that is changed only by “multiply by the forget gate, add what the input gate lets in”. That gives information a route across time with no weight matrix and no squashing on it. How well the route works is decided by the forget gate: near 1 it carries a number across a thousand steps, at its plain default of about 0.5 it forgets as fast as an RNN. The LSTM turns Day 59's problem from a fixed property of the weights into a dial the network can learn.

- Two states: c (memory) and h (what is shown). Three gates: f (keep), i (write), o (show). One candidate g.
- c_new = f * c_old + i * g; h_new = o * tanh(c_new). All four parts come from one matrix product.
- f = 1, i = 0 copies the memory forward exactly; o = 0 hides it without losing it.
- A gate that is 99.3% open loses the memory within a thousand steps; 99.995% keeps 95.6% of it.
- Direct route = product of forget gates: bounded by 1, about 0.5 to the power T at forget bias 0.
- Start the forget bias at 1 or 2. LSTMs can still explode through the indirect route: keep clipping.
- Four times the weights of an RNN of the same size.
- A sensitivity near 1 is not proof of memory: check which route it came by.

Suggested watch order, if starting from zero

- 1. Long Short-Term Memory (LSTM), Clearly Explained (StatQuest with Josh Starmer) — the cell built up piece by piece, with numbers.
- 2. Illustrated Guide to LSTM's and GRU's: A step by step explanation (The AI Hacker) — an animated walk through the gates; also previews the GRU (Day 66).

Day 64 preview

- **LSTM forward pass from scratch.** Today's `lstm_step_forward` goes inside a loop over time, as Day 57 did for the RNN cell, carrying both h and c from step to step and storing what the backward pass will need.
- Week 10 follows Week 9's pattern: each day adds a short exercise script, and Day 69 (sequence-to-sequence) is built from them.